In this Python code:

has_resources = all([resource.id in [r.id for r in nc[night_idx].resources] for resource in obs.required_resources()])

ChatGPT 5.2

recreated for each resource in obs.required_resources() when doing the check?

Yes, Sebastian, in the Python code you provided, the list comprehension `[r.id for r in nc[night_idx].resources]` is indeed recreated for each resource in `obs.required_resources()` during the execution of the `all()` function. This is because the list comprehension is inside the `all()` function, which iterates over each element in the outer list comprehension.

Here's a breakdown of the code:

- `obs.required_resources()` : This presumably returns a list (or an iterable) of resources that are required.
- `[r.id for r in nc[night_idx].resources]` : This list comprehension creates a list of `id` s from the resources available on a specific night (indexed by `night_idx` in `nc`).
- The outer list comprehension `[resource.id in [r.id for r in nc[night_idx].resources] for resource in obs.required_resources()]` checks for each required resource if its `id` is in the list of `id` s from the resources available on that night.

This means that for every `resource` in `obs.required_resources()` , the list `[r.id for r in nc[night_idx].resources]` is created anew, which can be inefficient, especially if `nc[night_idx].resources` or `obs.required_resources()` is large.

A more efficient approach would be to create the list of `id`s once before the `all()` function and then check if each required resource `id` is in this pre-created list:

python Copy code

```python
available_resource_ids = {r.id for r in nc[night_idx].resources}
has_resources = all(resource.id in available_resource_ids for resource in
```

By using a set comprehension `{r.id for r in nc[night_idx].resources}` instead of a list comprehension, we also gain a speedup in membership testing (`resource.id in available_resource_ids`), as checking membership in a set is generally faster than in a list.